

Phase 2: Requirement Analysis

Solution Requirements Specification (SRS)

1. Functional Requirements (FR)

Req ID	Requirement Title	Detailed Description	Verification Method
FR-1	User Input Capture	The system must provide an accessible web form capturing: Story Prompt, Character Name, Setting, Story Tone, and Art Style.	Manual Form Entry & Automated Test
FR-2	5-Panel Outline Generation	The system must call Gemini Flash (models/gemini-1.5-flash) to generate a structured 5-panel comic outline including panel numbers, titles, scene descriptions, and image prompts.	Schema Validation of JSON output
FR-3	Story & Dialogue Expansion	The system must invoke Gemini Pro (models/gemini-1.5-pro) using the panel outline to synthesize engaging comic narration, character dialogues, and ambient captions.	Narrative Text Inspection
FR-4	Panel Illustration Generation	The system must generate visual comic illustrations for each of the 5 panels based on AI-generated image prompts, storing them safely in static/panels/.	File Existence & Image Validity Check
FR-5	Layout Assembly & Binding	The system must integrate panel images, narrative text, scene context, and titles into a structured layout dictionary (build_comic_layout()).	Data Structure Unit Testing
FR-6	Multi-Page PDF Compilation	The system must compile all panels into a professionally formatted multi-page PDF document using FPDF with custom fonts, margins, and page stamping.	PDF Generation & Page Count Check
FR-7	Interactive Web Preview	The web application must display the generated comic strip sequentially with styled panel headers, speech bubbles, captions, and narrative boxes.	Browser DOM Verification
FR-8	RESTful API Endpoint	The system must expose a JSON endpoint (/generate-comic/json) accepting a Pydantic PromptRequest payload and returning layout data and PDF path.	HTTP POST TestClient Verification

2. Non-Functional Requirements (NFR)

NFR ID	Category	Requirement Detail	Metric / Target
NFR-1	Performance & Concurrency	Image generation for the 5 panels must execute concurrently using multi-threading to minimize user waiting latency.	End-to-end generation under 15 seconds.
NFR-2	Reliability & Resilience	The application must feature intelligent fallback handlers so that missing API keys or network hiccups never crash the server.	100% route uptime and zero unhandled 500 errors.
NFR-3	Usability & Aesthetics	The user interface must employ modern CSS styling, comic-book typography, glassmorphism, responsive grids, and animated loading feedback.	Google Lighthouse Accessibility & Best Practices > 90.
NFR-4	Modularity & Reusability	Core functionalities must be isolated into dedicated modules (gemini_flash.py, gemini_pro.py, image_generator.py, layout_builder.py, exporters.py, routes.py).	PEP 8 compliance, single-responsibility principle.
NFR-5	Data Integrity & Security	File paths and prompt strings must be sanitized to prevent directory traversal or malformed file writes.	Filename hashing and path validation.
NFR-6	Portability	The application must execute seamlessly across macOS, Linux, and Windows operating systems with zero OS-specific hardcoded dependencies.	Verified across standard Python 3.10+ environments.